

Deterministic where it matters. Fluid everywhere else.

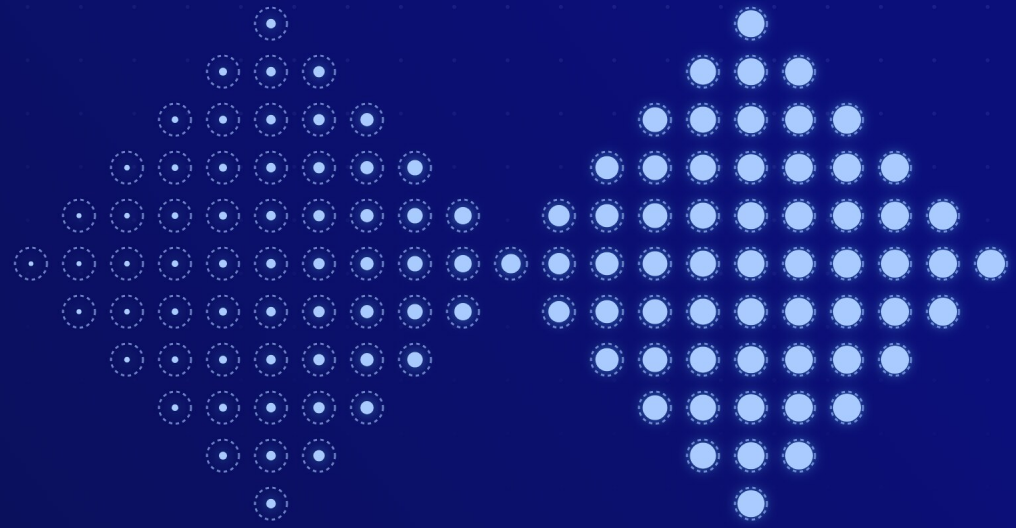
Designing trust into agentic developer tools.



Jordan Wright · Principal Product Designer, Salesforce · Apollo GraphQL panel

I design the conceptual model, then build it in the final medium to prove it holds.

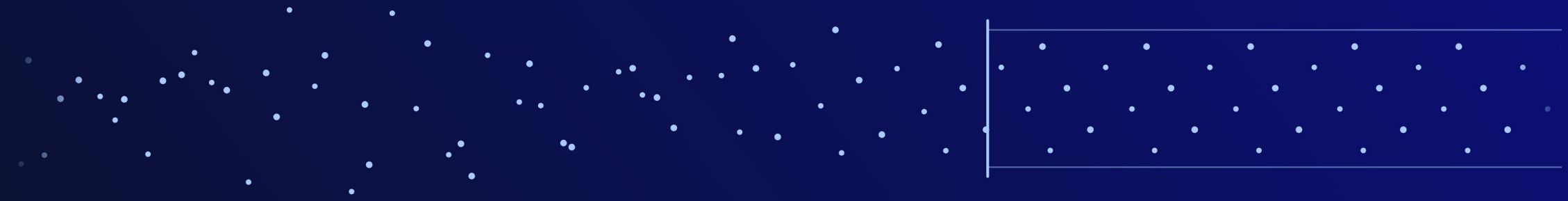
> On agent products the hard problem isn't the screen. It's **the model of what the agent may do** and **how a person stays in control.**



Jordan Wright · Principal Product Designer, Salesforce

When an agent can do the work, how does a person stay in control without slowing it down?

> I need to ‘approve’ a lot, I just clicked without reading the texts too much... coz I want to get the job done!
>
> VIBES CUSTOMER, Q1 2026



FREE

HELD · SAME SPEED

01

Agentforce Vibes



How much the agent may *do*, and where a person signs off.

02

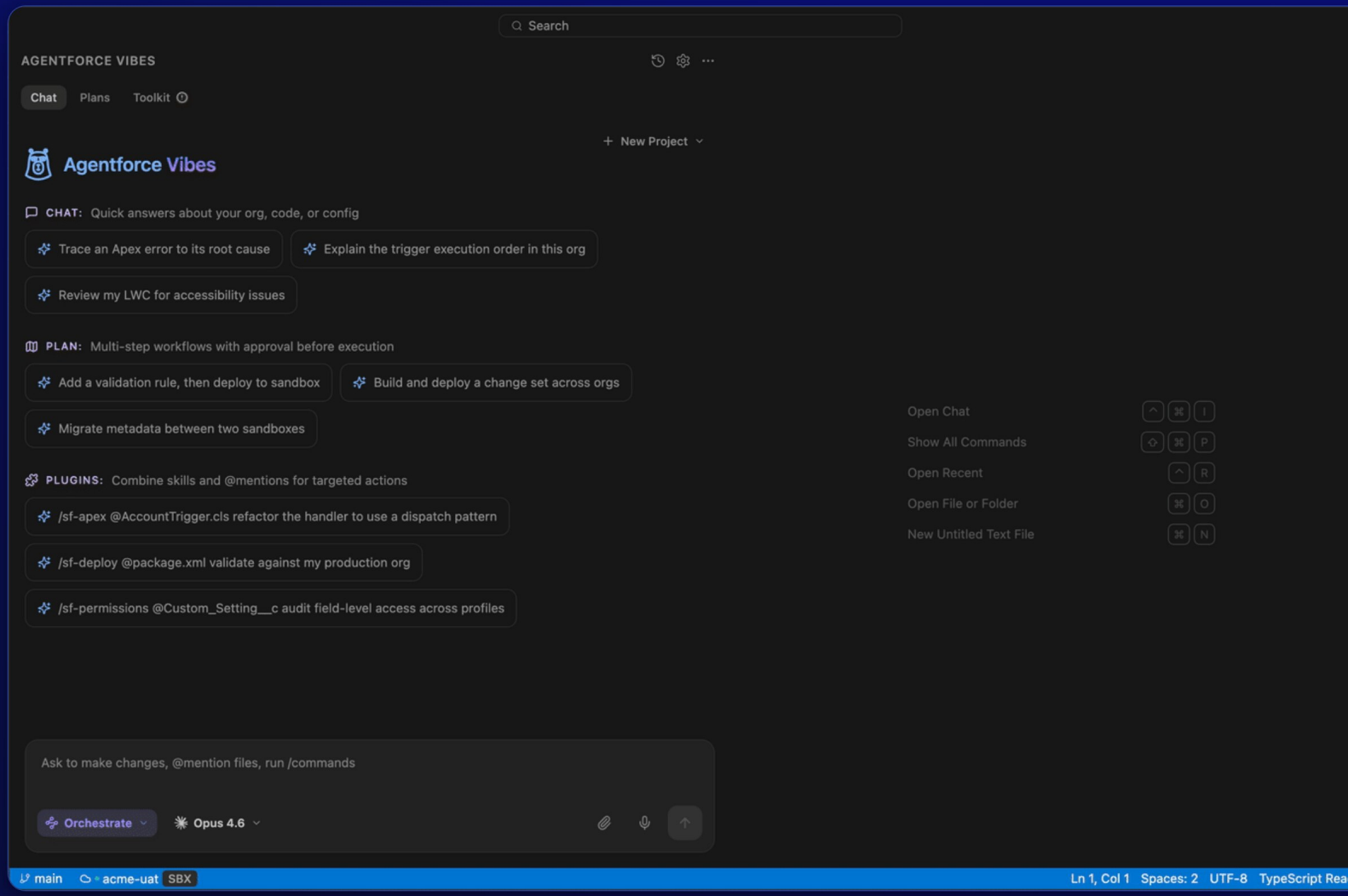
Salesforce Development Plugin for Claude Code



What the agent *can* do, and where you are.

Agentforce Vibes

The agentic coding chat for Salesforce developers. From a quick turn-by-turn fix to orchestrated subagents, the agent does the work and a person decides what ships.



Setup and tension

SETUP

Business need

Own the agent harness.
Vibes 1.0 was coupled to Cline's roadmap.

Monthly users fell **46%**

42K in Sep 2025 → 22.7K in May 2026



TENSION

The TDX demo

- A reactive response, stood up fast under pressure.
- Built for the stage, not daily use.

User need

Senior, multi-org Salesforce developers

- Move fast
- Understand what's happening
- Stay in control

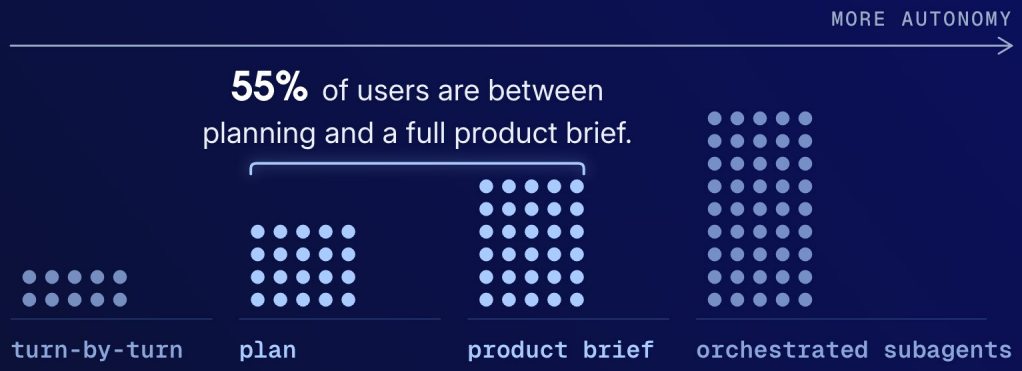
The real problem

It runs deeper than any one screen.

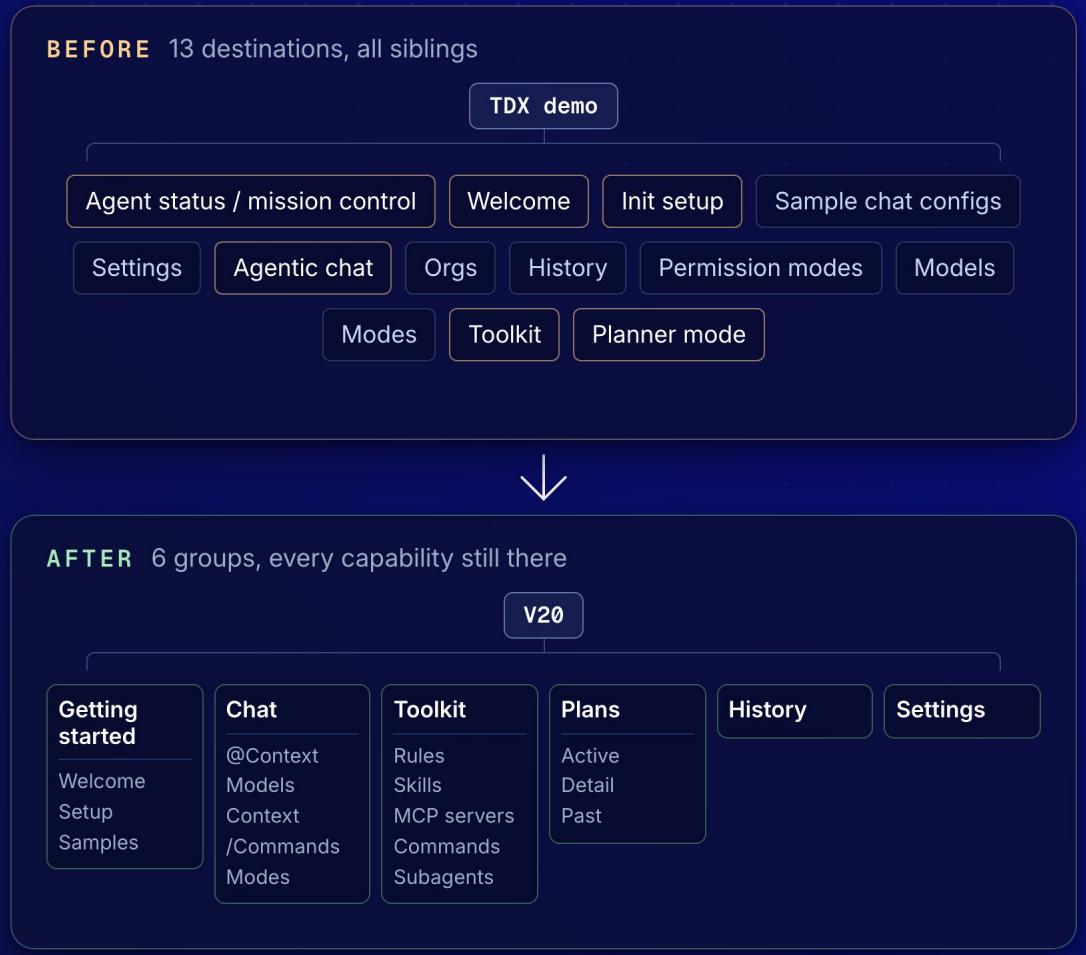
How it all connects, so a person can see what the agent is doing and where they hold the power to interject.

I took the demo apart into one flat list of concepts, then rebuilt the information architecture around control and **progressive enhancement**: you discover capabilities as you need them.

Scale of engagement: Autonomy grows with the user's ambition.



Speed inside a bounded workspace, friction only at irreversible changes. Nothing deploys without a human.



The call I made and rejected

Rejected

The TDX demo's structure, even with stage momentum behind it.

What I was after



One month to ship 1.0 right, instead of a buggy GA a week out.

How I got it

Prove it in real, working code, not a Figma file.

Intuitive IA: every capability still there, just reorganized.

V20: a prototype to incubate the new patterns.

Storybook: those patterns as working components.

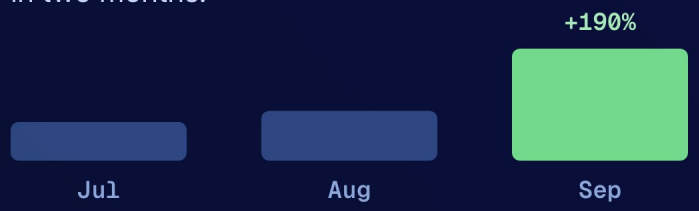
SVP of Platform Engineering, early May: "I want this." The direction changed.

What changed

+190%

customer usage, July → September

Free tier closed: −97%. Then usage nearly tripled in two months.



+93% customer developers

+50% requests per developer

+159% developers in paying accounts

1.0 in a month



> Agentforce Vibes... wasn't a tool I would choose as my primary AI assistant. **That has changed.**
>
> DEVELOPER REVIEW, JUNE 2026

It spread

- Other teams adopted vibes-ui (Metadata Visualizer, Select-to-Edit) and it reached the Dreamforce stage.

Salesforce Development Plugin for Claude Code

Access the power of the Salesforce platform right in the terminal, headlessly. It guides you through what you can do, where you're at, and what to try next, without leaving the agent.

Going headless loses twenty years of quiet UI affordances that taught people what they can do, where they are, and how the platform works.

The question: how do you create that for the user, just in time?



Setup and tension

SETUP

Business need

Make the platform's breadth usable from frontier coding agents.
Product teams keep adding skills.

115+ skills across the plugin

37 in this bundle

Too many skills, and the context window trims them.



TENSION

The existing proof of concept

Only worked for experts.

- You had to already be in an SFDX project.
- You had to already know the right Salesforce vocabulary.

User need

Without leaving the agent:

- See what's possible
- Where I am
- What's next

The real problem

Two gates kept non-experts out.

GATE 1

A frictionless way in.

Flow straight into building on Salesforce, headlessly.

GATE 2

Knowing what's possible.

Clear wayfinding, and a smart next step.

Out of that came my interaction model: **Fixed / Guided / Free**, each chosen by trading off the risk of being wrong against the risk of being noisy.

RISK OF BEING WRONG ····· RISK OF BEING NOISY



Fixed

Must be recognizable at a glance or have zero chance of hallucination. Org and project are always shown the same way; deterministic paints identical every time.



Guided

Low risk but complex. The agent gets context, and the user gets a predictable layout.



Free

Low risk and light. The agent picks up intent without signposting, because false positives get noisy fast.

The call I made and rejected

GATE 1 · A FRICTIONLESS WAY IN

REJECTED

“you need an SFDX project before you can start.”

```
› I want to build a form that saves to a custom object
✓ Creating your project: MyProject
🧩 Recommended for this job: LWC + Apex
```

✓ BUILT FREE GUIDED FIXED

Natural onboarding. Install the plugin and just start talking about what you want to do. It guides you to the right project for the job.

GATE 2 · KNOWING WHAT’S POSSIBLE

REJECTED

“developers don’t need wayfinding.”

```
org: acme-dev ✓ · Developer Edition
sfdx project: MyProject · main ✓ clean
🚀 Try next: Local source never deployed to this org.
```

✓ BUILT FIXED GUIDED

The Fixed / Guided / Free model. Claude Code lifecycle hooks render deterministic Python paints, so your org, your project, and **what’s possible always show up the same way.**

Follow-through: I retired my own six-stage journey rail. Development isn’t linear, and two stages could never be truthfully reached.

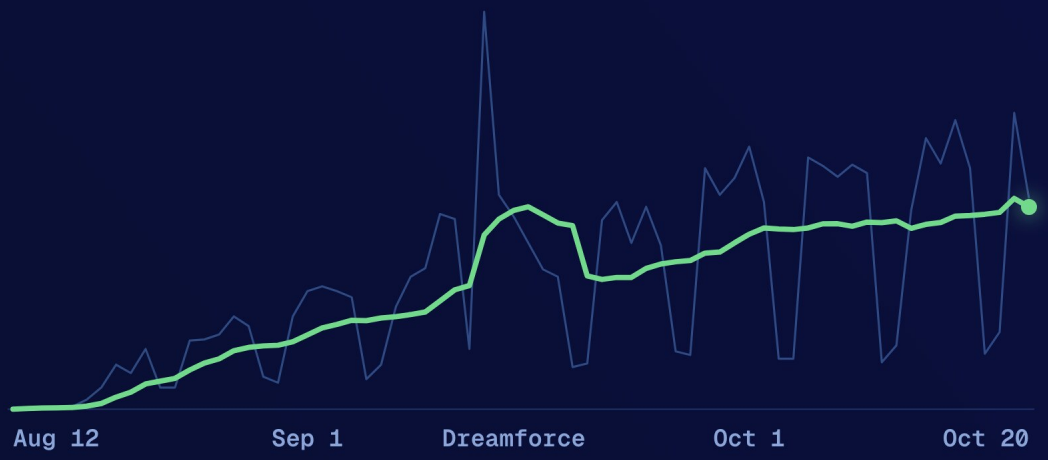
It became confidence-scored journey hints: one nudge at a time, low-confidence ones never shown.



What changed

GA in the official Claude Plugin Marketplace within 10 days (Aug 10).

~7× daily developers since GA. It spiked around Dreamforce, then kept climbing.



> The most useful thing I have added to my Salesforce workflow this year is not in Salesforce. **It is a plugin for Claude Code that Salesforce wrote themselves.**
>
> SALESFORCE DEVELOPER, LINKEDIN, SEPT 2026

GATE 1 · A FRICTIONLESS WAY IN

> The plugin … picks up your default org automatically. **Less setup and copy-pasting, more time building.**
>
> SALESFORCE DEVELOPER, LINKEDIN, OCT 2026

GATE 2 · KNOWING WHAT'S POSSIBLE

> The plugin now recognises when a task needs more than its core skills and **recommends a specialised plugin.**
>
> SALESFORCE DEVELOPER, LINKEDIN, OCT 2026

01

Decompose, then recompose.

Break it down to its atomic parts and rebuild it on design principles.

TDX demo → flat concepts → new IA

atoms/molecules/organisms

115+ skills → capability map → Free/Guided/Fixed

02

Design in the final medium.

Prove it, then ship it. Handshakes, not handoffs.

V20 → Storybook → "I want this" → npm packages *are* the Vibes front end

spike → PR #1030 as product code

03

Let evidence overrule me.



Reversed my own no-export policy the next day when it broke teammates

retired my own rail

> This release also stands out... towards AI native team with UX team working in lock step with engineers and being part of development cycle unlike traditional approach... we need to socialize your approach more and proliferate the practice across other product area.
>
> VP, SOFTWARE ENGINEERING

Deterministic where it matters. Fluid everywhere else.

In these two products, four things kept a person in control:



See what's possible

before you have to know what to ask for.



Know where you are:

which org, which project, what's next.



Understand what needs you:

autonomy can grow, but irreversible actions wait for a human.



See what happened:

every change, every step, on the record.

Agents can do the work. Design makes it something people can trust.